JAPAC AI BUILDER CUP 2026 · SUBMISSION

CarbonPilot

An AI carbon decision engine. It turns fragmented operational data into prioritised, financially-aware decarbonisation actions — and every number it prints is traceable to a calculation engine, never to the language model.

Measure → **Diagnose** → **Simulate** → **Act**

`rakazo-civo.tail1b51ed.ts.net:8443`

Working prototype · Gemini-driven agent over a deterministic engine · six screens, one loop

THE PROBLEM

Companies can already measure their carbon. They still cannot decide what to do about it.

01 **The data exists, the decision does not.** Electricity, cloud, travel, fleet and procurement sit in CSVs and ERP exports. Aggregating them produces a number, not a plan.

02 **Reporting is a backward view.** A footprint tells you where you were. Management asks what changes next — and what it will cost.

03 **Carbon and money are decided separately.** The largest reduction is often the most expensive, and nobody can see the trade-off in one place.

IN THE DEMO

DEMO FOOTPRINT

11,208

tCO_2e / yr

SCOPE 3 SHARE

61.1

%

Knowing you emit eleven thousand tonnes does not tell you what to do next.

Demo company: 168 activity records, 12 months, across electricity, cloud, travel, fleet and procurement.

THE MOTIVATING FINDING

The same load is 3,036 tonnes here — and 399 in New Zealand. Location is a decision variable, not a footnote.

LOAD

4,304,333

kWh / yr

INDONESIA

3,036

tCO_2e · rank 19/20 · coal 69%

NEW ZEALAND

399

tCO_2e

01 **Benchmarked, not estimated.** Twenty JAPAC grids from Ember Yearly Electricity Data (CC BY 4.0), one documented basis, labelled explicitly.

02 **Grid choice outweighs most single actions.** No reporting tool surfaces this — it sits in a footnote of the accounting exercise today.

Basis: full-lifecycle CO_2e intensity per kWh, Ember Yearly Electricity Data (CC BY 4.0). The product labels basis on every row; benchmark and inventory bases are never averaged together.

WHAT IT DOES

It answers the question after the footprint.

01

Ingest

A CSV is mapped to canonical activity types and scopes. The interpretation is shown for confirmation before anything is written.

02

Diagnose

A robust median/MAD test finds the months that moved, then attributes each one to the activities that moved with it.

03

Simulate

Interventions are applied to the stored baseline; a constraint search explores combinations of them.

04

Act

Recommendations are ranked on carbon × cost × effort, weighted by factor confidence, and tracked to an owner.

Six screens, one loop: each stage is its own engine and its own screen.

THE ONE ARCHITECTURAL RULE

The language model never produces a carbon number.

The engine computes. The agent explains.

Every kilogram of CO_2 is computed by a deterministic, unit-tested engine in app/engine/. The model chooses which analyses to run and explains the result — so its failure mode is "I could not find that", never "here is a plausible number".

Every recommendation shows its arithmetic

the quantity, the factor, the factor's source and year, and the uncertainty. A judge can ask why a factor is defensible and get a cited answer, not a recollection.

THE INTERACTION WORTH SHOWING

“Reduce our footprint 15% without increasing operating cost.”

One sentence in. A constraint search over 2,098 scenario combinations out. Real result, from the deployed instance:

CARBON REDUCTION	CARBON SAVED	COST IMPACT	FEASIBLE SCENARIOS
33.8	3,790	−$775	342
%	tCO_2e / yr	K / yr	of 2,098

WHAT THE ENGINE PUT IN THE PLAN

Renewable PPA for purchased electricity	80%
Mandate rail over short-haul flights	45%
Electrify the delivery fleet	40%
Supplier engagement and low-carbon sourcing	10%

Asked for 15%. It returned a plan that clears the carbon constraint and saves money — and can say why each action is in it.

Constraint search over a stored baseline; every scenario in the result satisfies both constraints by construction (tested).

An agent that orchestrates tools, over an engine that cannot be argued with.

13 tools — and the first turn is forced to call one. The model cannot answer a quantitative question from its own head.

Cited retrieval. A pure-stdlib BM25 ranker pulls the methodology passages behind a claim, so “why is that factor defensible” is answered from a source of record.

One process. FastAPI serves the UI and the API on a single port: no CORS layer, no second deployment, no vendor onboarding.

Stack Next.js static export · FastAPI · SQLite · Gemini over a gateway key · i18n EN/ID with no extra dependency.

Deployment One systemd unit behind a Tailscale Funnel today; Cloud Run / Firebase is the judging-path deployment, kept to a single command.

Honest scope No compliance reporting (CSRD, SB 253, CDP), no supplier portal or ERP connectors, curated point factors, linear scenario maths — stated, not discovered.

Stated boundaries, not avoided ones — the honesty a judge should hear before the room asks.

VERIFIED, NOT CLAIMED

105 tests. A smoke test that fails on a working-looking key. One process.

01 **Behaviour contracts, not snapshots.** A projection plus its saving equals the baseline. Constraint search only ever returns scenarios that satisfy the constraints. 105 tests, runnable without credentials.

02 **Preflight makes invisible failure visible.** A dead key produces no error — it produces a correct answer from the deterministic router while the UI still claims the model. The smoke test exists to catch exactly that.

03 **Every claim checkable.** pytest, tsc --noEmit, production build and preflight outputs are in the repository with the commands that produced them.

Verified 2026-10-04: backend 32s · frontend typecheck + static export ~1 min · preflight READY, 13 OK.

POSITION

From carbon accounting to carbon action.

THE CATEGORY, CONCEDED AND NARROWED

Today	CarbonPilot
Measure	Measure
Report	Diagnose
Analyse	Simulate
—	Act
Ranked by size	Ranked by carbon × cost × effort

Watershed, Persefoni, Sweep and SINAI already do accounting, ingestion and scenario planning. We do not claim to be the first AI carbon platform. The narrower claim: their AI targets the input pipeline — ours is a contract on the output.

Carbon reporting tells you where you are. CarbonPilot tells you what to do next.

Demonstration build — stated boundaries, not avoided ones.

Built for grids that make decarbonisation hard.

01 **A public facility, not a slide.** Twenty JAPAC grids, one documented basis, built on CC BY 4.0 open data — reusable by any team in the region.

02 **Bilingual EN/ID.** Reach into regional mid-market teams that reporting platforms under-serve.

03 **A pattern that generalises.** Wherever a model could plausibly invent a number, don’t let it. A deterministic engine with an explaining agent is the audit-grade shape for quantified AI.

GRIDS BENCHMARKED

20

JAPAC

LANGUAGES

2

EN / ID

TESTS GREEN

105

pytest

SCALING PATH

From CSV to the mid-market ERP.

01 **Connectors over CSVs.** AWS / GCP / Azure billing, Xero, SAP — per-account mapping on the same ingestion pipeline.

02 **Compliance modules on the same ledger.** CSRD, SB 253, CDP reports from the same audit trail, not a separate tool.

03 **Multi-tenancy and factor services.** Climatiq is wired and config-ready; isolation is the next engineering step, not an open question.

04 **Trivial ops by construction.** Static export + one process today; Cloud Run / Firebase is the scale path with no architectural change.

The boundaries in this deck are the honest ones: no compliance reporting yet, curated factors, linear scenario maths.

THE ASK

From carbon accounting to carbon action.

Measure → **Diagnose** → **Simulate** → **Act**

`rakazo-civo.tail1b51ed.ts.net:8443`

CarbonPilot — an AI carbon decision engine for JAPAC, built on Google AI, submitted to the AI Builder Cup 2026.

Prototype live · repository public at submission · proposal deck and 3-minute demo video included.